NAME - KRISH CHANDRASHEKHAR PACHODE ROLL NO. - B-18 BATCH - AIDS (SY-B)

AIM - To use sequence data types and their associated operations in Python such as Strings, Lists, Arrays, Tuples, Dictionaries, Sets, and Range, and to perform operations like indexing, slicing, insertion, deletion, updating, and traversal on these data structures.

THEORY -

1. What are sequence data types in Python?

Name them and explain how indexing and slicing work across different sequence types?

Sequence data types are collections of items arranged in a specific order. These elements can be accessed using their position (index).

Main sequence types in Python:

- List → `[1, 2, 3]`
- Tuple → `(1, 2, 3)`
- String → `"hello"`

Indexing

- Indexing means accessing elements using their position.
- Index starts from `0` (first element).
- Negative indexing starts from `-1` (last element).

Example:

```
a = [10, 20, 30, 40]
print(a[0])   # 10
print(a[-1])  # 40
```

Slicing

- Slicing means extracting a part of the sequence.
- Syntax: `sequence[start : end : step]`

Example:

```
a = [10, 20, 30, 40, 50]
print(a[1:4])   # [20, 30, 40]
```

```
print(a[:3])    # [10, 20, 30]
print(a[::2])   # [10, 30, 50]
```

---

2. What is the difference between a list and a tuple in Python?

Why would you prefer a tuple over a list in real-world applications?

Feature	List	Tuple
Mutability	Mutable (can change)	Immutable (cannot change)
Syntax	`[ ]`	`( )`
Performance	Slower	Faster
Use case	Dynamic data	Fixed data

Why prefer tuple?

- When data should not change (safety)
- Faster execution
- Can be used as dictionary keys (lists cannot)

Example:

```
t = (1, 2, 3)
# t[0] = 5  ❌ Error (immutable)
```

---

3. How are Python arrays different from lists?

When should you use the array module instead of a list?

Feature	List	Array (array module)
Data type	Can store different types	Stores same type only
Flexibility	High	Low
Performance	Slower	Faster for numeric data

When to use array?

- When working with large numerical data
- When memory efficiency is important

Example:

```
import array
arr = array.array('i', [1, 2, 3, 4])
```

---

4. Explain how dictionaries work internally in Python.

Why are dictionary lookups faster compared to lists or tuples?

- Python dictionaries use a **hash table** internally.

- Each key is passed through a **hash function** to get an index.
- Value is stored at that index.

Why fast lookup?

- Direct access using hash → O(1) time complexity
- No need to search entire data like lists

Example:

```
d = {"name": "Krish", "age": 20}
print(d["name"])   # Direct lookup
```

---

5. What is the difference between sets and lists in Python?

How do sets handle duplicate values, and where are sets useful in practical scenarios?

Feature	List	Set
Order	Ordered	Unordered
Duplicates	Allowed	Not allowed
Syntax	`[ ]`	`{ }`

How sets handle duplicates?

- Automatically remove duplicate values.

Example:

```
s = {1, 2, 2, 3}
print(s)   # {1, 2, 3}
```

Practical uses of sets:

- Removing duplicates from a list
- Membership testing (fast)
- Mathematical operations (union, intersection)

Example:

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a & b)  # Intersection → {2, 3}
```

```
# 1. String length and first three characters (len, slice)
s = "Hello Python"
print("Q1 Output:")
print(len(s))        # len() function
```

```
print(s[0:3])        # slicing
print()



# 2. List max and min (max, min)
lst = [10, 5, 25, 8, 15]
print("Q2 Output:")
print(max(lst))      # max() function
print(min(lst))      # min() function
print()



# 3. Tuple display using loop (enumerate)
students = ("Aman", "Riya", "Karan", "Neha")
print("Q3 Output:")
for i, name in enumerate(students):  # enumerate() function
    print(i, name)
print()



# 4. Dictionary keys (keys function)
d = {101: "Aman", 102: "Riya", 103: "Karan"}
print("Q4 Output:")
print(list(d.keys()))   # keys() + list()
print()



# 5. Remove duplicates (set function)
lst = [1, 2, 2, 3, 4, 4, 5]
print("Q5 Output:")
unique = set(lst)       # set() function
print(unique)
print()



# 6. Count vowels (count function alternative)
s = "Hello Python"
print("Q6 Output:")
vowel_count = sum(s.count(v) for v in "aeiouAEIOU")  # count() + sum()
print(vowel_count)
print()



# 7. List insertion and deletion (append, insert, pop, del)
lst = [10, 20, 30, 40]
print("Q7 Output:")

lst.append(50)        # append()
lst.insert(1, 15)     # insert()
```

```
print("After insertion:", lst)

lst.pop()             # pop()
del lst[0]            # del keyword
print("After deletion:", lst)
print()



# 8. Conversion (tuple, list functions)
print("Q8 Output:")
lst = [1, 2, 3]
t = tuple(lst)        # tuple()
print(t)

tup = (4, 5, 6)
lst2 = list(tup)      # list()
print(lst2)
```

```
Q1 Output:
12
Hel

Q2 Output:
25
5

Q3 Output:
0 Aman
1 Riya
2 Karan
3 Neha

Q4 Output:
[101, 102, 103]

Q5 Output:
{1, 2, 3, 4, 5}

Q6 Output:
3

Q7 Output:
After insertion: [10, 15, 20, 30, 40, 50]
After deletion: [15, 20, 30, 40]

Q8 Output:
(1, 2, 3)
[4, 5, 6]
```

Conclusion

In this experiment, we used various Python built-in functions such as len(), max(), min(), enumerate(), keys(), set(), count(), sum(), append(), insert(), pop(), tuple(), and list() to perform different operations on data structures. This helped in understanding

how different functions simplify coding and improve efficiency. Overall, the experiment provided practical knowledge of using multiple Python functions in real programs.